

Teacher:	Ray Joshua Balingkit	**Subject:**	Computer Science
Week:	5 (Term 1)	**Year:**	10-11
Link:		**Cover:**	

LESSON OVERVIEW

What should students know/do by the end of this week's lessons?

Students will develop their understanding of arithmetic operators and expressions in Python. They will evaluate calculations using operator precedence and parentheses and explain the purposes of integer division and remainder. Students will then build a calculator using numeric inputs and labelled output. They will predict results, compare them with program output and use normal, zero and decimal values to identify and correct calculation errors.

End-of-Week Assessment/Check: How will you know they got it?

Teacher questioning, discussion of worked examples, the completed student tasks and the final checks in both lessons. Compare hand calculations with program results and explain one use of %. Check three test cases and ask students to diagnose a deliberately incorrect expression. Check that students can explain their reasoning and apply the skill without relying on the worked example.

LESSON SLOTS - MONDAY & FRIDAY

Lesson	Date	Learning Objective	The Lesson Flow	Differentiation
1	21 September 2026	By the end of this lesson, students will be able to evaluate arithmetic expressions using operator precedence.	**H:** Predict the difference between 3 + 4 * 2 and (3 + 4) * 2. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain arithmetic operators, parentheses, integer division and remainder. Explain addition, subtraction, multiplication and division before comparing /, // and %. Discuss how parentheses change the order of evaluation and why the program must use numeric values when performing arithmetic. **WD:** Evaluate and run expressions for sharing items and calculating a remainder. Calculate 3 + 4 * 2 and (3 + 4) * 2 by hand, then compare the Python results. Share 17 items between 5 people	**Support:** Provide an operator reference, a worked precedence example and a table for predictions. Use physical counters for integer division and remainder, then help students translate the calculation into an expression before working independently. **Extension:** Students investigate calculations involving zero and decimal values and explain why division by zero cannot produce a normal numeric result. They compare two equivalent expressions and justify where parentheses improve readability.

			and explain how // gives the whole-number share while % gives the number left over. **YD:** Write a program that calculates a basket total and shares a whole number of items equally. Students record expressions and predicted results before running them. Their basket program uses clear variable names and labelled totals; their sharing calculation reports both the whole-number share and the remaining items. **C:** Compare hand calculations with program results and explain one use of %. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
2	25 September 2026	By the end of this lesson, students will be able to build and test a calculator with numeric inputs.	**H:** Present a calculator that joins digits instead of adding them. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Review conversion, clear variable names and labelled output. Explain how numeric conversion, arithmetic expressions and clear output work together in a calculator. Discuss the need to specify which operations are supported and to test the requirements rather than checking only whether the program runs. **WD:** Plan and code a two-input calculator for sum, difference and product. Plan two numeric inputs, convert them using float(), and calculate their sum,	**Support:** Provide an input-process-output planning sheet and a partially completed calculator. Model the sum calculation, then ask students to write the difference and product expressions. Use a test table with one completed example before removing the scaffold. **Extension:** Students add division with a clear zero-divisor check using a supplied decision structure, or describe the necessary check in plain language. They justify test values that reveal string concatenation or an incorrect arithmetic operator.

			difference and product. Predict the results for 4 and 2, then change one input to a decimal and compare the output with a hand calculation. **YD:** Complete the calculator and test positive, zero and decimal values; record expected and actual results. Students build and save the calculator, using meaningful identifiers and labelled output for each operation. They record expected and actual results for positive, zero and decimal inputs and correct an expression that fails a test. **C:** Check three test cases and ask students to diagnose a deliberately incorrect expression. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
3				Support: Extension:
4				Support: Extension:

MATERIALS NEEDED FOR THE WEEK

- Pearson Edexcel International GCSE Computer Science Student Book: Chapter 5
- Teacher computer and classroom display
- Classroom computers with Python 3 editor and individual save folders
- Prepared algorithm, tracing or Python practice tasks aligned with the lesson objectives
- Whiteboard and markers
- Student notebooks and pencils
- Success criteria, test tables and teacher observation checklist



END-OF-WEEK REFLECTION

- What worked well?

- What flopped or confused students?

- What needs to be re-taught or adjusted next week?